

Design Justification Report

INT8 MAC Accelerator for CNN Inference

ECE 410/510 HW4AI Spring 2026 | Dawood Alrandi

1. Problem and Motivation

Neural network inference on edge devices is bottlenecked by the convolution operation. Profiling in M1 showed that the `Conv2D_im2col` kernel accounts for 69.6% of total runtime on an Apple M1 CPU, with an arithmetic intensity of 1.06 FLOP per byte. The M1 ridge point is 38.1 FLOP per byte, meaning the kernel is deeply memory-bandwidth-bound. The CPU compute units sit idle more than 95% of the time waiting for data. General-purpose processors are fundamentally mismatched to this workload because they were not designed for the tight repeated multiply-accumulate loop at the heart of convolution. A dedicated hardware accelerator that keeps the compute units fed with data can achieve far better energy efficiency for this specific workload.

2. Roofline Analysis

The target hardware platform is the SkyWater sky130 PDK synthesized at 100 MHz with a PCIe register-map interface providing 0.1 GB per second effective bandwidth. The hardware ridge point is 0.2 GFLOP/s divided by 0.1 GB/s which equals 2.0 FLOP per byte. My kernel arithmetic intensity bounds are 0.989 FLOP per byte (no reuse lower bound) to 1.957 FLOP per byte (weight-stationary reuse upper bound). Both bounds fall below the hardware ridge point of 2.0 FLOP per byte, placing the kernel in the memory-bound region of the HW roofline. Compared to the M1 software baseline which also sits at 1.06 FLOP per byte, the architectural conclusion is the same: both are memory-bound, but the hardware design uses 57000 times less power. The bottleneck is the PCIe register-map interface bandwidth, not the compute units.

3. Precision and Data Format

I chose INT8 for weights and activations with an INT32 accumulator. INT8 uses 4 times less memory bandwidth than FP32, which raises arithmetic intensity by 4x compared to a FP32 baseline. The quantization error analysis in CF04 showed a mean absolute error of 0.0043 using symmetric per-tensor quantization with scale factor $S = \max \text{ of absolute weight values divided by } 127$. This is below the 1% threshold for acceptable quantization error per published baselines. The INT32 accumulator prevents overflow for up to 2^{23} accumulations, covering any practical layer size. No floating point hardware is needed in the compute core, which reduces area and power.

4. Dataflow and Architecture

The accelerator uses a weight-stationary dataflow. Weights are loaded once into the interface registers and remain fixed while activations stream through one per clock cycle. Each clock cycle computes one multiply-accumulate operation: the activation value a times the weight value b , sign-extended to 32 bits, added to the accumulator. The compute core is a single-lane pipelined MAC unit with registered output. The interface module wraps the compute core with a register-map protocol. This dataflow minimizes

weight memory traffic by loading each weight once and reusing it for all activations in the batch, which is the weight-stationary pattern. The data path is: host writes activation and weight pair to register 0x00, valid_in pulses high for one cycle, compute core multiplies and accumulates, host reads result from register 0x08.

5. Hardware Interface

The interface uses a PCIe-style AXI4-Lite register map. The host writes 32-bit data to address 0x00 to trigger a MAC operation, where bits 7 to 0 carry the activation and bits 15 to 8 carry the weight. Address 0x04 resets the accumulator. Address 0x08 returns the 32-bit accumulated result. The interface is implemented as interface_mod.sv which instantiates compute_core.sv internally. Effective bandwidth at 100 MHz with one register write per MAC is 100 million writes per second times 2 bytes per write which equals 200 MB per second. The actual effective rate is lower due to host overhead per transaction. The design is interface-bound: the compute core can accept one MAC per cycle but the host overhead limits throughput. Quantified: effective throughput is approximately 100 MFLOP per second against a compute capability of 200 MFLOP per second.

6. Verification

The design was verified at three levels. In M2, compute_core.sv and interface_mod.sv were tested independently with unit testbenches. The compute core testbench applied 9 MAC operations representing a 3x3 convolution kernel with inputs 5 3 2 1 4 2 1 3 2 and weights 3 negative 2 1 4 negative 1 2 negative 3 1 2, computing an expected output of 19 verified by hand calculation. In M3, an end-to-end co-simulation testbench drove the integrated top module through the interface only, writing a equals 5 b equals 3 and a equals 2 b equals 4, reading back 23, and verifying against the independent hand calculation 5 times 3 plus 2 times 4 equals 23. In M4, the final simulation repeats the M3 test and shows PASS in final_run.log. All testbenches use always_ff with signed arithmetic and sign extension to ensure correct negative number handling.

7. Synthesis Results

The design was synthesized with OpenLane 2 targeting sky130_fd_sc_hd at 100 MHz (10 ns clock period). Total cell area is 521.3 square micrometers with 112 cells. The compute core accounts for 79 percent of area at 412.5 square micrometers and the interface module accounts for 21 percent at 108.8 square micrometers. The worst negative slack is negative 0.31 ns, meaning the critical path takes 10.31 ns, which misses the 10 ns target by 0.31 ns. The critical path runs from the weight register through the AND2 binary multiply chain to the ADDDF adder tree and ends at the output accumulator flip-flop. There is one hold violation on the reset path with a slack of negative 0.04 ns. The effective achievable clock is 96.9 MHz. Power estimation gives 0.175 mW total at the typical process corner.

8. Benchmark Results

The M1 software baseline measured 8.43 ms execution time for 234752 MACs at 27800 GFLOP per second. The M4 hardware accelerator projects 2.35 ms execution time at 0.1 GFLOP per second effective throughput, limited by the register-map interface. The throughput speedup is 0.1 divided by 27800 equals 0.0000036x, meaning the hardware is slower in raw throughput. However the power comparison tells a different story: M1 chip at approximately 10 W versus hardware core at 0.175 mW, a 57000x power reduction. Energy per MAC for the hardware is 0.175 mW divided by 100 MHz equals

1.75 picojoules per MAC. The gap in throughput is explained entirely by the interface bottleneck. With a DMA burst interface loading 256 pairs per transaction, projected throughput would reach 25 GFLOP per second, matching the M1 at 57000x lower power.

9. What Did Not Work

Three things did not work as planned. First, the PCIe register-map interface creates a severe throughput bottleneck. I designed the interface for correctness and simplicity in M2, but did not account for host transaction overhead. The fix is a burst DMA interface, which was planned for M4 but not implemented due to time constraints. Second, the timing closure at 100 MHz failed by 0.31 ns. The critical path through the ripple-carry adder tree is longer than expected. Replacing the ADDF ripple-carry chain with a carry-save adder would reduce this path by about 2.5 ns and allow 130 MHz operation. Third, the power estimation used default 10 percent toggle rates from OpenSTA rather than workload-realistic rates from the profiled kernel. This means the 0.175 mW estimate is approximate. The actual power depends on how often the inputs change, which for a streaming convolution workload would be close to 100 percent toggle rate per cycle, potentially doubling the switching power estimate.